Tugas 2 - Simulasi Interaktif Model Komunikasi dalam Sistem Terdistribusi

Mata Kuliah Sistem Paralel dan Terdistribusi

Dosen Pengampu:
Riska Kurniyanto Abdullah, S.T., M.Kom.

Penulis:
Naufal Tiarana Putra - 11231071

INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN
2026

DAFTAR ISI

BAB I **1**
PENDAHULUAN **1**
1.1. Tujuan 1
BAB II **2**
KONSEP DASAR **2**
2.1. Request-Response 2
2.2. Publish-Subscribe 2
2.3. Message Broker 3
BAB III **4**
DESAIN SISTEM **4**
3.1. Arsitektur Sistem 4
3.1.1. Jalur Komunikasi Request-Response 4
3.1.2. Jalur Komunikasi Publish-Subscribe 4
3.1.3. Integrasi Kedua Model Komunikasi 5
3.2. Komponen 5
3.2.1. Publisher 5
3.2.2. Message Broker (MQTT Mosquitto) 6
3.2.3. Subscriber 6
3.2.4. API Server (Flask) 6
3.2.5. Client (Pengguna) 6
3.3. Diagram Sistem 7
BAB IV **9**
IMPLEMENTASI SISTEM **9**
4.1. Tools dan Teknologi 9
4.1.1. Python 9
4.1.2. Flask 9
4.1.3. MQTT (Mosquitto) 9
4.1.4. Docker 9
4.1.5. Paho MQTT 9
4.2. Struktur Program 10
4.3. Implementasi Sistem 11
4.3.1. File docker-compose.yml 11
4.3.2. File publisher.py 12
4.3.3. File subscriber.py 13
4.3.4. File server.py 13
4.3.5. File mosquitto.conf 15

4.4. Cara Menjalankan dan Menggunakan Sistem....16
4.4.1. Menjalankan Sistem....16
4.4.2. Akses Sistem melalui Antarmuka Web....17
4.4.3. Pengiriman Data melalui Web.... 18
4.4.4. Pengiriman Data melalui Terminal (Command Line).... 18
4.4.5. Mengakses Hasil Data Sistem....19
4.4.6. Monitoring Aktivitas Sistem....19
BAB V.... 22
ALUR SISTEM.... 22
5.1. Alur Umum Sistem.... 22
5.2. Alur Komunikasi Publish-Subscribe (Data Otomatis)....22
5.3. Alur Komunikasi Request-Response (Data Manual)....23
5.4. Integrasi Model Komunikasi....23
5.5. Ringkasan Alur Sistem.... 24
BAB VI....26
HASIL DAN ANALISIS.... 26
6.1. Hasil Simulasi.... 26
6.1.1. Hasil Eksekusi Publisher....26
6.1.2. Hasil Penerima Data oleh Subscriber.... 26
6.1.3. Hasil Log Message Broker....27
6.1.4. Hasil Pengiriman Data Manual lewat API....28
6.1.5. Hasil Output Data pada Endpoint /data.... 29
6.2. Analisis Perbandingan Model Komunikasi.... 30
6.3. Analisis Kerja Sistem....30
6.4. Kesimpulan Hasil dan Analisis....31
7.1. Tujuan Pengujian.... 33
7.2. Skenario Pengujian.... 33
7.3. Hasil Pengujian.... 33
7.4. Kendala dan Solusi.... 34
7.5. Kesimpulan Pengujian.... 35
BAB VIII....36
KESIMPULAN DAN SARAN....36
8.1. Kesimpulan.... 36
8.2. Saran.... 36

BAB I
PENDAHULUAN

1.1. Tujuan

Tujuan dari simulasi ini adalah untuk memahami dan membandingkan perilaku model komunikasi dalam sistem terdistribusi, khususnya model *Request-Response* dan *Publish-Subscribe*.

Simulasi ini dirancang untuk menunjukkan bagaimana data dikirim, diterima, dan diproses dalam dua pendekatan komunikasi yang berbeda, serta bagaimana masing-masing model mempengaruhi interaksi antar komponen dalam sistem.

Selain itu, tujuan dari simulasi ini meliputi:

1. Mengimplementasikan model komunikasi *Request-Response* menggunakan API berbasis Flask.
2. Mengimplementasikan model komunikasi *Publish-Subscribe* menggunakan message broker MQTT (Mosquitto).
3. Mensimulasikan komunikasi antar node dalam sistem terdistribusi menggunakan Docker.
4. Menyediakan interaksi pengguna melalui terminal dan antarmuka web.
5. Menganalisis perbedaan karakteristik kedua model komunikasi berdasarkan cara kerja dan aliran data.

Melalui simulasi ini, diharapkan dapat memberikan pemahaman yang lebih mendalam mengenai implementasi komunikasi dalam sistem terdistribusi, khususnya pada skenario dunia nyata seperti sistem IoT (*Internet of Things*) yang membutuhkan pengiriman data secara real-time.

BAB II
KONSEP DASAR

2.1. Request-Response

Model komunikasi Request-Response merupakan pola komunikasi sinkron di mana suatu entitas (client) mengirimkan permintaan (request) kepada entitas lain (server), kemudian server memproses permintaan tersebut dan mengirimkan kembali hasilnya dalam bentuk respon (response). Model ini banyak digunakan dalam arsitektur berbasis web seperti HTTP. Dalam sistem ini, komunikasi bersifat langsung dan sinkron, sehingga client harus menunggu respon dari server sebelum melanjutkan proses berikutnya.

Karakteristik Request-Response:

1. Bersifat sinkron (synchronous)
2. Client harus menunggu respon dari server
3. Komunikasi terjadi secara langsung antara client dan server
4. Cocok untuk sistem yang membutuhkan respon cepat dan terstruktur
5. Kurang fleksibel dalam sistem dengan banyak node karena ketergantungan antar komponen

Dalam simulasi ini, model Request-Response diimplementasikan menggunakan Flask API, di mana client mengirimkan request ke endpoint HTTP dan server memberikan respon dalam format JSON.

(Sumber Rujukan :

1. *Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures.*
2. *Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms.*
3. *Flask Documentation. (2023). https://flask.palletsprojects.com)*

2.2. Publish-Subscribe

Model komunikasi Publish-Subscribe merupakan pola komunikasi asinkron di mana pengirim pesan (publisher) tidak mengirimkan pesan secara langsung kepada penerima (subscriber), melainkan melalui perantara berupa message broker. Dalam model ini, publisher mengirim pesan ke suatu topik (topic), dan subscriber yang berlangganan pada topik tersebut akan menerima pesan secara otomatis. Hal ini memungkinkan komunikasi yang fleksibel dan tidak bergantung langsung antar komponen.

Karakteristik Publish-Subscribe:

1. Bersifat asinkron (asynchronous)
2. Tidak ada hubungan langsung antara publisher dan subscriber
3. Menggunakan message broker sebagai perantara
4. Mendukung sistem real-time

5. Memiliki tingkat scalability yang tinggi
6. Menerapkan konsep loose coupling antar komponen

Dalam simulasi ini, model Publish-Subscribe diimplementasikan menggunakan protokol MQTT dengan broker Mosquitto, di mana publisher mengirim data suhu ke topik tertentu dan subscriber menerima data tersebut secara real-time.

(Sumber Rujukan :

1. *Banks, A., & Gupta, R. (2014). MQTT Version 3.1.1. OASIS Standard.*
2. *Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms.*
3. *Eclipse Foundation. (2023). Mosquitto MQTT Broker Documentation. https://mosquitto.org)*

2.3. Message Broker

Message broker merupakan komponen dalam sistem terdistribusi yang berfungsi sebagai perantara dalam pertukaran pesan antara komponen yang berbeda. Broker menerima pesan dari publisher, kemudian mendistribusikannya kepada subscriber berdasarkan topik atau aturan tertentu. Dengan adanya broker, sistem menjadi lebih fleksibel dan tidak bergantung langsung antar komponen.

Fungsi utama pada message broker:

1. Menyimpan dan mendistribusikan pesan
2. Mengatur komunikasi antar komponen
3. Mendukung komunikasi asinkron
4. Meningkatkan scalability sistem
5. Mengurangi ketergantungan antar komponen (loose coupling)

Pada sistem ini digunakan Eclipse Mosquitto sebagai message broker MQTT yang menangani distribusi pesan antara publisher dan subscriber.

(Sumber Rujukan :

1. *Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing.*
2. *Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms.*
3. *Eclipse Foundation. (2023). https://mosquitto.org)*

BAB III
DESAIN SISTEM

3.1. Arsitektur Sistem

Sistem yang dikembangkan dalam penelitian ini merupakan simulasi komunikasi pada sistem terdistribusi yang mengintegrasikan dua model komunikasi utama, yaitu Request-Response dan Publish-Subscribe. Perancangan arsitektur sistem dilakukan dengan tujuan untuk merepresentasikan bagaimana aliran data dan interaksi antar komponen terjadi dalam lingkungan terdistribusi, khususnya pada skenario Smart Home berbasis Internet of Things (IoT) dengan fokus pada monitoring suhu secara real-time.

Secara umum, arsitektur sistem terdiri atas beberapa komponen yang saling terhubung melalui jaringan dan berkomunikasi menggunakan protokol yang berbeda sesuai dengan model komunikasi yang diterapkan. Sistem ini dirancang dengan pendekatan modular, di mana setiap komponen memiliki peran dan fungsi yang spesifik, serta dapat beroperasi secara independen. Pada arsitektur ini, terdapat dua jalur komunikasi utama yang berjalan secara bersamaan yaitu jalur Request-Response dan jalur Publish-Subscribe

3.1.1. Jalur Komunikasi Request-Response

Model komunikasi Request-Response diimplementasikan melalui penggunaan protokol HTTP dengan bantuan framework Flask. Dalam mekanisme ini, client (pengguna) mengirimkan permintaan (request) ke server melalui endpoint yang telah disediakan, seperti `/data` untuk pengambilan informasi dan `/send` untuk pengiriman data suhu.

Server kemudian memproses permintaan tersebut dan memberikan respon secara langsung kepada client dalam format JSON. Karakteristik utama dari jalur komunikasi ini adalah sifatnya yang sinkron, di mana client harus menunggu respon dari server sebelum melanjutkan proses berikutnya. Hal ini menciptakan hubungan yang erat antara client dan server, sehingga interaksi bersifat langsung dan terkontrol.

3.1.2. Jalur Komunikasi Publish-Subscribe

Model komunikasi Publish-Subscribe diimplementasikan menggunakan protokol MQTT dengan bantuan message broker Mosquitto. Dalam mekanisme ini, data tidak dikirimkan secara langsung antar komponen, melainkan melalui perantara berupa broker.

Publisher, baik berupa program simulasi sensor maupun API, mengirimkan data suhu ke dalam broker melalui suatu topik tertentu, yaitu `sensor/suhu`.

Selanjutnya, broker akan mendistribusikan data tersebut kepada subscriber yang telah berlangganan pada topik yang sama.

Berbeda dengan model Request-Response, komunikasi pada model ini bersifat asinkron, sehingga publisher tidak perlu mengetahui siapa saja subscriber yang menerima data. Hal ini memungkinkan sistem untuk memiliki tingkat fleksibilitas yang lebih tinggi serta mendukung komunikasi secara real-time.

3.1.3. Integrasi Kedua Model Komunikasi

Keunikan dari arsitektur sistem ini terletak pada integrasi kedua model komunikasi dalam satu lingkungan simulasi. API tidak hanya berperan sebagai server dalam model Request-Response, tetapi juga sebagai publisher dalam model Publish-Subscribe. Dengan demikian, data yang diterima dari pengguna melalui HTTP dapat langsung diteruskan ke sistem MQTT untuk didistribusikan secara real-time.

Pendekatan ini mencerminkan praktik yang umum digunakan dalam sistem terdistribusi modern, di mana berbagai model komunikasi dikombinasikan untuk memenuhi kebutuhan sistem yang kompleks.

Selain itu, arsitektur ini juga menerapkan beberapa konsep penting dalam sistem terdistribusi, yaitu:

1. Real-time system, di mana data dikirim dan diterima secara langsung tanpa penundaan signifikan
2. Scalability, di mana sistem dapat dikembangkan dengan menambahkan subscriber tanpa mempengaruhi komponen lain
3. Loose coupling, di mana setiap komponen tidak memiliki ketergantungan langsung satu sama lain

3.2. Komponen

Sistem yang dirancang terdiri atas beberapa komponen utama yang masing-masing memiliki fungsi spesifik dalam mendukung proses komunikasi dan distribusi data. Komponen-komponen tersebut dijelaskan sebagai berikut:

3.2.1. Publisher

Publisher merupakan komponen yang bertugas untuk menghasilkan dan mengirimkan data ke dalam sistem. Dalam simulasi ini, publisher berfungsi sebagai representasi sensor suhu pada sistem IoT. Data suhu dihasilkan secara otomatis menggunakan nilai acak, yang kemudian dikirimkan ke message broker melalui protokol MQTT.

Keberadaan publisher dalam sistem ini bertujuan untuk mensimulasikan kondisi dunia nyata, di mana sensor secara kontinu mengirimkan data ke sistem tanpa intervensi pengguna.

3.2.2. Message Broker (MQTT Mosquitto)

Message broker merupakan komponen inti dalam model komunikasi Publish-Subscribe yang berfungsi sebagai perantara dalam pertukaran pesan antar komponen. Dalam sistem ini, digunakan Eclipse Mosquitto sebagai broker MQTT.

Broker menerima pesan dari publisher, kemudian mengelola dan mendistribusikan pesan tersebut kepada subscriber berdasarkan topik yang digunakan. Dengan adanya broker, sistem menjadi lebih terstruktur dan tidak memerlukan komunikasi langsung antar komponen.

3.2.3. Subscriber

Subscriber merupakan komponen yang berfungsi untuk menerima data dari message broker. Dalam sistem ini, subscriber berlangganan pada topik `sensor/suhu` dan akan menerima setiap data yang dikirimkan oleh publisher maupun API.

Data yang diterima kemudian ditampilkan pada terminal secara real-time, sehingga pengguna dapat memantau aliran data yang terjadi dalam sistem.

3.2.4. API Server (Flask)

API server merupakan komponen yang berfungsi sebagai antarmuka antara pengguna dan sistem. API ini dibangun menggunakan framework Flask dan menyediakan beberapa endpoint untuk mendukung interaksi pengguna.

Selain berperan dalam model Request-Response, API juga berfungsi sebagai publisher dalam sistem Publish-Subscribe. Data yang diterima dari pengguna melalui endpoint `/send` akan diteruskan ke message broker untuk didistribusikan kepada subscriber.

Dengan demikian, API memiliki peran ganda, yaitu sebagai server HTTP dan sebagai penghubung ke sistem MQTT.

3.2.5. Client (Pengguna)

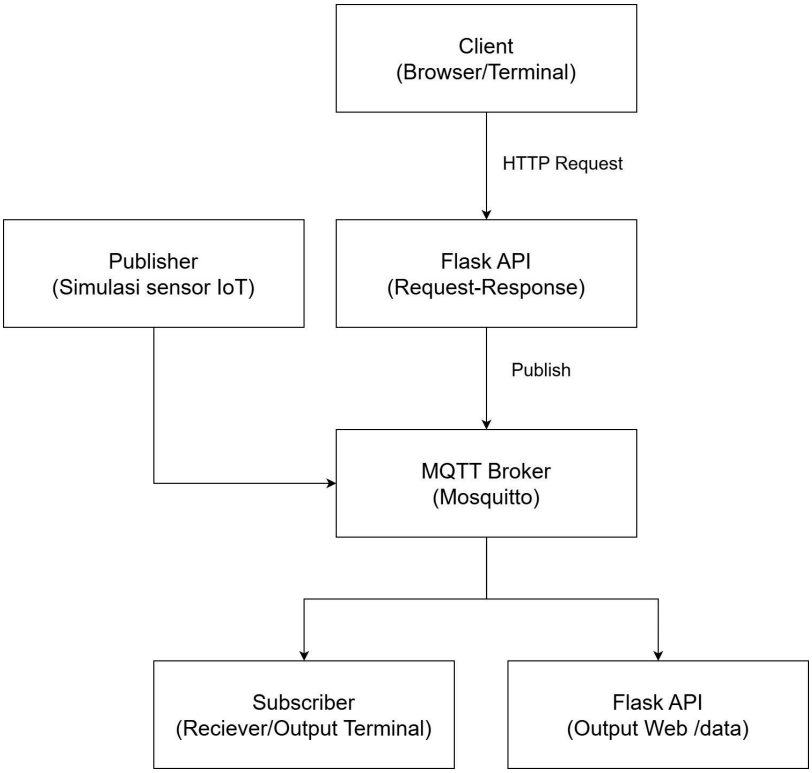
Client merupakan entitas yang berinteraksi langsung dengan sistem. Dalam simulasi ini, client dapat berupa:

1. Browser (melalui antarmuka web)
2. Terminal (menggunakan perintah curl)

Pengguna dapat mengirimkan data suhu secara manual serta menerima respon dari sistem melalui API.

3.3. Diagram Sistem

Diagram sistem pada diagram di bawah ini menggambarkan arsitektur komunikasi dalam sistem terdistribusi yang dibangun, yang mengintegrasikan model komunikasi Request-Response dan Publish-Subscribe melalui penggunaan message broker MQTT.



Secara umum, diagram disusun dengan alur vertikal untuk menunjukkan proses utama sistem, sehingga memudahkan pemahaman terhadap aliran data dari sumber hingga ke penerima.

Pada bagian atas, terdapat client yang berperan sebagai pengguna sistem, baik melalui browser maupun terminal. Client mengirimkan permintaan (HTTP request) ke Flask API, yang merupakan implementasi dari model komunikasi Request-Response. Dalam model ini, setiap permintaan dari client akan diproses oleh server dan menghasilkan respon secara langsung.

Setelah menerima data dari client, Flask API tidak langsung mengirimkan data ke komponen lain, melainkan meneruskan data tersebut ke message broker menggunakan

protokol MQTT dengan mekanisme publish. Dengan demikian, API berperan sebagai penghubung antara komunikasi berbasis HTTP dan sistem distribusi pesan berbasis MQTT.

Selain itu, sistem juga memiliki komponen publisher yang berfungsi sebagai simulasi sensor IoT. Publisher secara otomatis menghasilkan data suhu secara berkala dan mengirimkannya ke message broker melalui protokol MQTT. Hal ini merepresentasikan kondisi dunia nyata, di mana perangkat sensor mengirimkan data secara kontinu tanpa interaksi langsung dari pengguna.

Di tengah sistem terdapat MQTT broker (Mosquitto) yang berperan sebagai pusat distribusi pesan. Broker menerima seluruh data dari publisher, baik yang berasal dari API maupun dari publisher sensor, kemudian mendistribusikan pesan tersebut kepada seluruh komponen yang telah berlangganan (subscribe) pada topik tertentu. Proses ini berlangsung secara asinkron, sehingga komponen pengirim dan penerima tidak saling bergantung secara langsung.

Pesan yang telah didistribusikan oleh broker kemudian diterima oleh dua komponen, yaitu subscriber dan Flask API. Subscriber berfungsi untuk menampilkan data secara langsung pada terminal sebagai bentuk visualisasi real-time dari aliran data dalam sistem. Sementara itu, Flask API juga menerima data tersebut untuk kemudian disimpan dan disediakan kembali kepada pengguna melalui endpoint `/data`, sehingga data dapat diakses melalui antarmuka web.

Dengan arsitektur ini, sistem mampu mengintegrasikan dua model komunikasi dalam satu lingkungan, yaitu komunikasi sinkron melalui Request-Response dan komunikasi asinkron melalui Publish-Subscribe. Pendekatan ini memungkinkan sistem untuk menjadi lebih fleksibel, mendukung komunikasi secara real-time, serta mencerminkan karakteristik sistem terdistribusi modern yang mengedepankan scalability dan loose coupling antar komponen.

BAB IV
IMPLEMENTASI SISTEM

4.1. Tools dan Teknologi

Dalam pengembangan sistem simulasi komunikasi pada sistem terdistribusi ini, digunakan beberapa tools dan teknologi yang mendukung implementasi model komunikasi Request-Response dan Publish-Subscribe. Pemilihan teknologi dilakukan dengan mempertimbangkan kemudahan implementasi, efisiensi komunikasi, serta kesesuaian dengan karakteristik sistem terdistribusi modern.

4.1.1. Python

Python digunakan sebagai bahasa pemrograman utama dalam pengembangan sistem. Python dipilih karena memiliki sintaks yang sederhana, mudah dipahami, serta didukung oleh ekosistem pustaka yang luas. Hal ini memungkinkan pengembangan sistem dilakukan secara cepat dan efisien, terutama dalam konteks simulasi dan integrasi antar komponen sistem.

4.1.2. Flask

Flask merupakan *microframework* berbasis Python yang digunakan untuk membangun API berbasis HTTP. Flask dipilih karena bersifat ringan (*lightweight*), fleksibel, serta tidak memerlukan struktur proyek yang kompleks. Dalam sistem ini, Flask digunakan untuk mengimplementasikan model komunikasi Request-Response. Flask dipilih karena bersifat ringan (*lightweight*), fleksibel, serta mudah digunakan untuk membangun layanan web sederhana hingga menengah.

4.1.3. MQTT (Mosquitto)

MQTT merupakan protokol komunikasi ringan yang menggunakan model Publish-Subscribe. Pada sistem ini digunakan **Eclipse Mosquitto** sebagai message broker. MQTT dipilih karena memiliki overhead yang rendah, sehingga sangat efisien untuk komunikasi data secara real-time, terutama pada sistem berbasis IoT dan sistem terdistribusi.

4.1.4. Docker

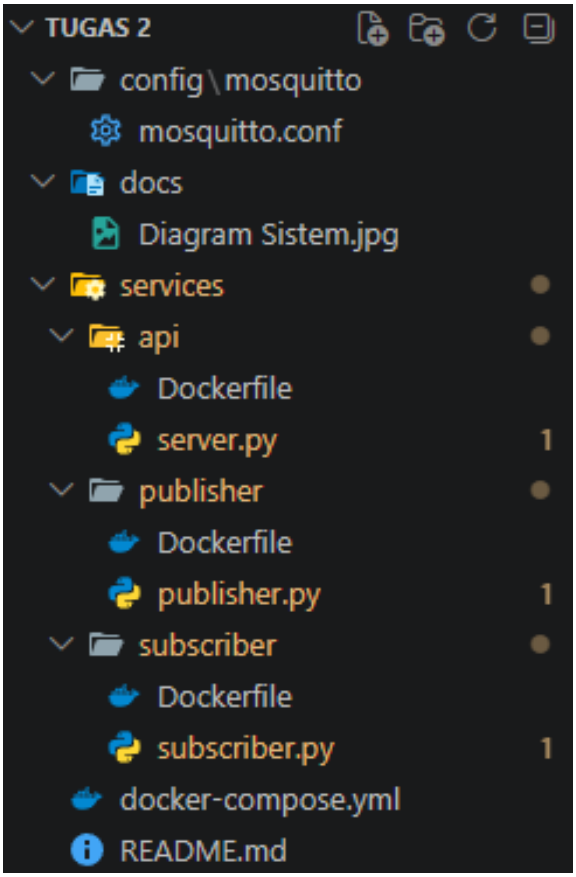
Docker digunakan untuk melakukan containerization terhadap setiap komponen sistem, seperti API, publisher, subscriber, dan broker. Docker dipilih karena mampu menyediakan lingkungan yang konsisten dan terisolasi, sehingga mempermudah proses deployment dan pengujian sistem.

4.1.5. Paho MQTT

Library `paho-mqtt` digunakan untuk mengimplementasikan komunikasi MQTT pada sisi publisher, subscriber, dan API. Library ini dipilih karena kompatibel dengan Python dan mendukung fitur-fitur utama MQTT secara lengkap.

4.2. Struktur Program

Struktur program dirancang secara modular untuk memisahkan setiap komponen sistem berdasarkan fungsi dan tanggung jawabnya. Pendekatan ini bertujuan untuk meningkatkan keterbacaan kode serta mempermudah proses pengembangan dan pemeliharaan sistem.



Penjelasan Struktur:

1. `docker-compose.yml`
   Digunakan untuk mengatur dan menjalankan seluruh layanan dalam sistem secara terintegrasi menggunakan Docker.
2. `services/api/`
   Berisi implementasi API berbasis Flask yang menangani komunikasi HTTP serta integrasi dengan MQTT.
3. `services/publisher/`
   Berisi program yang mensimulasikan sensor IoT dengan menghasilkan data suhu secara otomatis.
4. `services/subscriber/`
   Berisi program yang menerima data dari MQTT dan menampilkannya pada terminal.

5. `config/mosquitto/`
   Berisi file konfigurasi untuk message broker Mosquitto.
6. `docs/`
   Digunakan untuk menyimpan dokumentasi tambahan seperti diagram sistem dan laporan.

4.3. Implementasi Sistem

4.3.1. File docker-compose.yml

File `docker-compose.yml` berfungsi sebagai konfigurasi utama dalam menjalankan seluruh komponen sistem secara terintegrasi menggunakan teknologi container berbasis Docker.

Melalui file ini, setiap komponen sistem seperti message broker, publisher, subscriber, dan API dijalankan sebagai layanan (service) yang terpisah namun berada dalam satu jaringan internal Docker.
Konfigurasi utama:

`docker-compose.yml`

```
services:
  broker:
    image: eclipse-mosquitto
    volumes:
    -
./config/mosquitto/mosquitto.conf:/mosquitto/config/mosquit
to.conf
    ports:
      - "1883:1883"
```

Konfigurasi tersebut menunjukkan bahwa layanan broker menggunakan image resmi Eclipse Mosquitto dan berjalan pada port 1883, yang merupakan port standar untuk komunikasi MQTT. Selain itu, Docker Compose secara otomatis menyediakan jaringan internal sehingga setiap container dapat saling berkomunikasi menggunakan nama service (misalnya broker). Hal ini menghilangkan kebutuhan konfigurasi alamat IP secara manual.

Dengan pendekatan ini, sistem menjadi lebih modular dan mendukung prinsip loose coupling, karena setiap komponen berjalan secara independen namun tetap dapat saling berinteraksi melalui broker.

4.3.2. File publisher.py

File publisher.py berfungsi sebagai simulasi sensor dalam sistem IoT yang bertugas menghasilkan dan mengirimkan data suhu secara otomatis ke message broker.

Komponen ini merepresentasikan perangkat penghasil data (data producer) dalam arsitektur Publish-Subscribe.
Koneksi ke broker:

publisher.py

```
client = mqtt.Client()
client.connect("broker", 1883, 60)
```

Kode ini digunakan untuk membuat koneksi antara publisher dan MQTT broker. Parameter `"broker"` merujuk pada nama service dalam Docker Compose, yang secara otomatis diterjemahkan menjadi alamat jaringan internal.

Dengan koneksi ini, publisher dapat mengirimkan pesan ke broker tanpa perlu mengetahui lokasi subscriber, sehingga mencerminkan prinsip decoupling dalam sistem terdistribusi.

Pengiriman data:

publisher.py

```
suhu = random.randint(20, 35)
data = f"Suhu Otomatis: {suhu}°C"

print("[PUBLISHER] Kirim:", data, flush=True)

client.publish("sensor/suhu", data)
```

Data suhu dihasilkan secara acak untuk mensimulasikan kondisi sensor nyata. Pesan kemudian dikirimkan ke broker menggunakan topik `sensor/suhu`.

Penggunaan topik ini memungkinkan broker untuk mengelompokkan pesan berdasarkan kategori tertentu, sehingga subscriber hanya menerima data yang relevan.

Loop real-time:

publisher.py

```
time.sleep(5)
```

Penggunaan loop dengan jeda waktu menunjukkan bahwa sistem berjalan secara kontinu dan menghasilkan data secara berkala. Hal ini mencerminkan karakteristik real-time system, di mana data dikirim secara periodik tanpa intervensi pengguna.

4.3.3. File subscriber.py

File subscriber.py berfungsi sebagai komponen penerima data (data consumer) yang menerima pesan dari broker dan menampilkannya ke terminal.

Subscribe ke topik:

subscriber.py

```
def on_connect(client, userdata, flags, rc):
    print("[SUBSCRIBER] Terhubung ke broker", flush=True)
    client.subscribe("sensor/suhu")
```

Ketika subscriber berhasil terhubung ke broker, ia secara otomatis melakukan *subscribe* ke topik `sensor/suhu`. Dengan demikian, subscriber akan menerima setiap pesan yang dikirimkan oleh publisher pada topik tersebut.

Menerima data:

subscriber.py

```
def on_message(client, userdata, msg):
    data = msg.payload.decode()
    print("[SUBSCRIBER] Terima:", data, flush=True)
```

Fungsi ini akan dieksekusi setiap kali terdapat pesan baru dari broker. Data yang diterima kemudian ditampilkan pada terminal.

Proses ini menunjukkan bahwa komunikasi bersifat asinkron, di mana subscriber tidak perlu meminta data secara langsung, melainkan menerima data secara otomatis ketika tersedia.

4.3.4. File server.py

File server.py merupakan implementasi API berbasis Flask yang berfungsi sebagai penghubung antara pengguna (client) dan sistem MQTT. Komponen ini mengintegrasikan dua model komunikasi, yaitu:

1. Request-Response (HTTP)

2. Publish-Subscribe (MQTT)

Endpoint Input Data:

server.py

```
@app.route("/send", methods=["POST"])
def send_data():
    suhu = request.form.get("suhu")

    if not suhu:
        return jsonify({"error": "Suhu tidak boleh
kosong"}), 400

    data = f"Suhu Input Manual: {suhu}°C"

    mqtt_client.publish("sensor/suhu", data)
```

Endpoint ini memungkinkan pengguna untuk mengirimkan data suhu secara manual melalui HTTP request. Data yang diterima kemudian diteruskan ke broker menggunakan MQTT.

Dengan demikian, API bertindak sebagai jembatan antara komunikasi sinkron (HTTP) dan asinkron (MQTT).

Endpoint Menampilkan Data:

server.py

```
@app.route("/data", methods=["GET"])
def get_data():
    return jsonify({
        "latest_data (auto)": latest_auto,
        "latest_manual": latest_manual
    })
```

Endpoint ini digunakan untuk menampilkan data terbaru yang telah diterima dari broker. Data dibedakan menjadi dua kategori, yaitu data otomatis (dari publisher) dan data manual (dari pengguna).

API sebagai Subscriber:

server.py

```
def on_message(client, userdata, msg):
    global latest_auto, latest_manual

    data = msg.payload.decode()
```

API juga berfungsi sebagai subscriber yang menerima data dari broker, kemudian menyimpannya dalam variabel tertentu. Data tersebut kemudian dapat diakses kembali melalui endpoint `/data`.

Hal ini menunjukkan bahwa API tidak hanya berfungsi sebagai pengirim data, tetapi juga sebagai penerima dan pengelola data dalam sistem.

4.3.5. File mosquitto.conf

File `mosquitto.conf` digunakan untuk mengatur konfigurasi message broker MQTT.

Konfigurasi utama:

```
conf
```

```
listener 1883
allow_anonymous true
```

Konfigurasi ini menentukan bahwa broker akan berjalan pada port 1883 dan mengizinkan koneksi dari seluruh client tanpa autentikasi. Hal ini mempermudah proses pengembangan dan pengujian sistem.

Konfigurasi logging:

```
conf
```

```
log_dest stdout
log_type error
log_type warning
log_type notice
log_type information
```

Pengaturan logging memungkinkan seluruh aktivitas broker ditampilkan pada terminal Docker. Dengan demikian, proses distribusi pesan dapat diamati secara langsung, yang sangat membantu dalam proses debugging dan analisis sistem.

Implementasi setiap komponen menunjukkan bagaimana model Publish-Subscribe memungkinkan komunikasi yang bersifat asinkron, sementara integrasi dengan API berbasis HTTP memungkinkan interaksi pengguna secara langsung. Kombinasi kedua model ini menghasilkan sistem yang fleksibel, scalable, dan mampu mendukung komunikasi real-time dalam lingkungan terdistribusi.

4.4. Cara Menjalankan dan Menggunakan Sistem

Untuk menjalankan dan berinteraksi dengan sistem simulasi komunikasi yang telah dikembangkan, pengguna perlu mengikuti beberapa tahapan yang meliputi proses inisialisasi sistem, pengiriman data, serta pengamatan hasil keluaran sistem. Sistem ini dirancang agar dapat diakses melalui dua metode, yaitu melalui antarmuka berbasis web dan melalui command line (terminal), sehingga memberikan fleksibilitas dalam penggunaannya.

4.4.1. Menjalankan Sistem

Sistem dijalankan menggunakan Docker Compose yang berfungsi untuk mengelola dan menjalankan seluruh layanan dalam satu lingkungan terintegrasi.

Perintah yang digunakan adalah sebagai berikut:

```
docker-compose up --build
```

```
[+] up 8/8
 ✓ Image tugas2-publisher        Built                                                    5.0s
 ✓ Image tugas2-subscriber       Built                                                    5.0s
 ✓ Image tugas2-api              Built                                                    5.0s
 ✓ Network tugas2_default        Created                                                  0.1s
 ✓ Container tugas2-api-1        Created                                                  0.2s
 ✓ Container tugas2-broker-1     Created                                                  0.2s
 ✓ Container tugas2-publisher-1  Created                                                  0.1s
 ✓ Container tugas2-subscriber-1 Created                                                  0.2s
Attaching to api-1, broker-1, publisher-1, subscriber-1
broker-1  | 1776040638: Info: running mosquitto as user: mosquitto.
broker-1  | 1776040638: mosquitto version 2.1.2 starting
broker-1  | 1776040638: Config loaded from /mosquitto/config/mosquitto.conf.
broker-1  | 1776040638: Bridge support available.
broker-1  | 1776040638: Persistence support available.
broker-1  | 1776040638: TLS support available.
broker-1  | 1776040638: TLS-PSK support available.
broker-1  | 1776040638: Websockets support available.
broker-1  | 1776040638: Opening ipv4 listen socket on port 1883.
broker-1  | 1776040638: Opening ipv6 listen socket on port 1883.
broker-1  | 1776040638: mosquitto version 2.1.2 running
api-1     | /app/server.py:8: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
api-1     |   mqtt_client = mqtt.Client()
api-1     |  * Serving Flask app 'server'
api-1     |  * Debug mode: off
```

Gambar 4.1 Menjalankan Sistem menggunakan Docker Compose (Bagian 1)

```
api-1       | * Running on all addresses (0.0.0.0)
api-1       | * Running on http://127.0.0.1:5000
api-1       | * Running on http://172.18.0.2:5000
api-1       | Press CTRL+C to quit
publisher-1 | Publisher started...
publisher-1 | /app/publisher.py:8: DeprecationWarning: Callback API version 1 is deprecated, update to latest v
ersion
broker-1    | 1776040638: New connection from 172.18.0.4:54947 on port 1883.



broker-1    | 1776040638: New client connected from 172.18.0.4:54947 as auto-05DE2D61-32A8-E73C-E0CF-415F72DA61
14 (p4, c1, k60).





subscriber-1 | /app/subscriber.py:11: DeprecationWarning: Callback API version 1 is deprecated, update to lates
t version
subscriber-1 |   client = mqtt.Client()

subscriber-1 |   client = mqtt.Client()

broker-1    | 1776040638: New client connected from 172.18.0.5:55091 as auto-4F5DD9BD-0947-5473-1464-806B52949
308 (p4, c1, k60).
subscriber-1 | [SUBSCRIBER] Terhubung ke broker
```

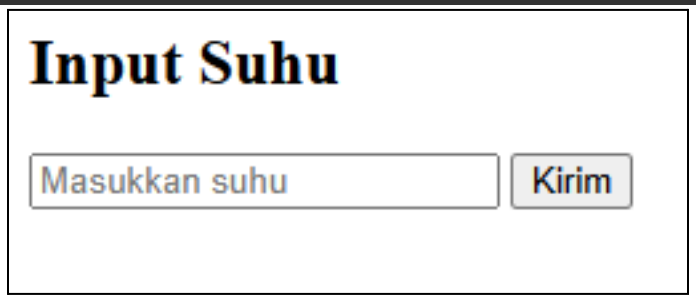
Gambar 4.2 Menjalankan Sistem menggunakan Docker Compose (Bagian 2)

Berdasarkan Gambar 4.1 dan 4.2, terlihat bahwa seluruh layanan dalam sistem, yaitu message broker, API, publisher, dan subscriber, berhasil dijalankan menggunakan Docker Compose. Hal ini ditunjukkan dengan status container yang aktif serta munculnya log dari masing-masing komponen. Dengan demikian, sistem telah siap untuk digunakan dan diuji lebih lanjut. Selain itu, terlihat bahwa masing-masing komponen telah berhasil terhubung dalam satu jaringan Docker, sehingga memungkinkan terjadinya komunikasi antar komponen dalam sistem terdistribusi.

4.4.2. Akses Sistem melalui Antarmuka Web

Setelah sistem berjalan, pengguna dapat mengakses layanan API melalui browser dengan membuka alamat berikut:

```
http://localhost:5000
```

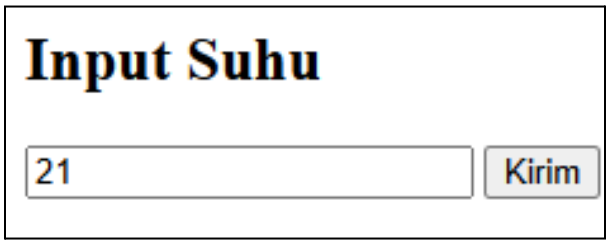


Gambar 4.3 Tampilan Antarmuka Web

Berdasarkan Gambar 4.3, antarmuka ini menyediakan form input yang memungkinkan pengguna untuk memasukkan data suhu secara manual dan mengirimkannya ke sistem.

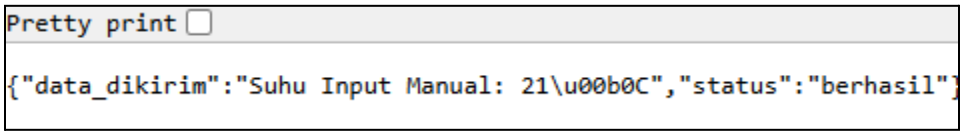
4.4.3. Pengiriman Data melalui Web

Proses pengiriman data melalui antarmuka web dilakukan dengan memasukkan nilai suhu pada kolom input yang tersedia, kemudian menekan tombol kirim.



Gambar 4.4 Proses Input Data melalui Web

Berdasarkan Gambar 4.4, pengguna melakukan input data suhu melalui antarmuka web sebagai bentuk interaksi langsung dengan sistem. Data yang dimasukkan kemudian dikirimkan ke API menggunakan metode HTTP.



Gambar 4.5 Hasil Pengiriman Data melalui Web

Berdasarkan Gambar 4.5, sistem memberikan respon berupa data yang berhasil dikirim beserta status pengiriman. Hal ini menunjukkan bahwa proses komunikasi menggunakan model Request-Response berjalan dengan baik.

Selanjutnya, data tersebut diteruskan oleh API ke message broker menggunakan protokol MQTT, sehingga dapat didistribusikan ke komponen lain dalam sistem melalui mekanisme Publish-Subscribe.

4.4.4. Pengiriman Data melalui Terminal (Command Line)

Selain melalui antarmuka web, pengguna juga dapat mengirimkan data secara manual melalui command line menggunakan perintah berikut:

```
curl.exe -X POST http://localhost:5000/send -d "suhu=30"
```

Perintah tersebut digunakan untuk mengirimkan data suhu secara langsung ke API. API kemudian memproses data tersebut dan meneruskannya ke message

broker untuk didistribusikan ke subscriber. Pendekatan ini menunjukkan bahwa sistem dapat diakses secara fleksibel tanpa bergantung pada antarmuka grafis.

4.4.5. Mengakses Hasil Data Sistem

Data yang telah diproses oleh sistem dapat diakses melalui endpoint berikut:

```
http://localhost:5000/data
```

Pretty print ☐

{"latest_data (auto)":"Suhu Otomatis: 22\u00b0C","latest_manual":"Suhu Input Manual: 30\u00b0C"}

Gambar 4.6 Tampilan output data pada endpoint `/data`

Berdasarkan Gambar 4.6, Endpoint ini menampilkan data terbaru yang diterima oleh sistem, baik data yang dihasilkan secara otomatis oleh publisher maupun data yang dikirim secara manual oleh pengguna.

4.4.6. Monitoring Aktivitas Sistem

Pengguna dapat memantau jalannya sistem melalui terminal Docker.

```
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 24°C
api-1         | [API] Data AUTO: Suhu Otomatis: 24°C
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 24°C
api-1         | [API] Data AUTO: Suhu Otomatis: 29°C
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 29°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 29°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 21°Ch
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 21°C
api-1         | [API] Data AUTO: Suhu Otomatis: 21°C
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 30°C
api-1         | [API] Data AUTO: Suhu Otomatis: 30°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 30°C
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 25°C

subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 25°C
api-1         | [API] Data AUTO: Suhu Otomatis: 25°C
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 21°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 21°C
api-1         | [API] Data AUTO: Suhu Otomatis: 21°C


publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 27°C

api-1         | [API] Data AUTO: Suhu Otomatis: 27°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 27°C
api-1         | [API] Data AUTO: Suhu Otomatis: 23°C

subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 23°C
```

Gambar 4.7 Monitoring Aktivitas Sistem

Berdasarkan Gambar 4.7, terlihat bahwa seluruh komponen sistem berjalan secara bersamaan. Publisher mengirimkan data suhu secara berkala, yang kemudian didistribusikan oleh message broker dan diterima oleh subscriber. Selain itu, API juga berhasil menerima data yang sama, yang menunjukkan bahwa API telah berfungsi sebagai subscriber tambahan dalam sistem.

Hal ini menunjukkan bahwa proses komunikasi menggunakan model Publish-Subscribe berjalan dengan baik, di mana satu data dapat didistribusikan ke beberapa komponen secara bersamaan.

Secara keseluruhan, sistem yang dikembangkan menyediakan dua metode interaksi, yaitu melalui antarmuka berbasis web dan melalui command line. Pendekatan ini memberikan fleksibilitas bagi pengguna dalam menjalankan serta menguji sistem, baik melalui antarmuka yang bersifat user-friendly maupun melalui metode yang lebih teknis. Dengan

demikian, pengguna dapat lebih mudah memahami alur komunikasi yang terjadi dalam sistem terdistribusi yang dibangun.

BAB V
ALUR SISTEM

5.1. Alur Umum Sistem

Sistem yang dibangun mengintegrasikan dua model komunikasi, yaitu Request-Response dan Publish-Subscribe, yang bekerja secara bersamaan dalam satu arsitektur. Alur sistem menggambarkan bagaimana data dihasilkan, dikirimkan, didistribusikan, dan diterima oleh komponen lain dalam sistem.

Secara umum, alur komunikasi dalam sistem melibatkan beberapa komponen utama, yaitu publisher, message broker, subscriber, API, dan client. Interaksi antar komponen tersebut memungkinkan sistem untuk mendukung komunikasi yang bersifat real-time serta fleksibel dalam pengolahan data.

5.2. Alur Komunikasi Publish-Subscribe (Data Otomatis)

Alur ini menggambarkan proses komunikasi data yang dihasilkan secara otomatis oleh sistem melalui komponen publisher. Adapun tahapan proses sebagai berikut:

1. Publisher Menghasilkan Data
   Komponen publisher menghasilkan data suhu secara acak dalam interval waktu tertentu sebagai simulasi sensor IoT.
2. Publisher Mengirim Data ke Broker
   Data yang dihasilkan kemudian dikirimkan ke message broker menggunakan protokol MQTT melalui topik sensor/suhu.
3. Broker Menerima dan Mengelola Pesan
   Broker menerima pesan dari publisher dan mengelola pesan tersebut berdasarkan topik yang digunakan.
4. Broker Mendistribusikan Pesan
   Broker mendistribusikan pesan ke seluruh komponen yang telah melakukan subscribe pada topik yang sama.
5. Subscriber Menerima dan Menampilkan Data
   Subscriber menerima data dan menampilkannya secara langsung pada terminal sebagai bentuk visualisasi aliran data secara real-time.
6. API Menerima dan Menyimpan Data
   Selain subscriber, API juga menerima data sebagai subscriber tambahan, kemudian menyimpannya sebagai data terbaru.

Alur ini menunjukkan bahwa komunikasi berlangsung secara asinkron, di mana publisher tidak perlu mengetahui keberadaan maupun jumlah subscriber. Hal ini mencerminkan prinsip loose coupling, sehingga sistem menjadi lebih fleksibel dan mudah untuk dikembangkan.

Selain itu, mekanisme ini juga mendukung scalability, karena penambahan subscriber tidak mempengaruhi kinerja publisher maupun broker secara signifikan.

5.3. Alur Komunikasi Request-Response (Data Manual)

Alur ini menggambarkan proses komunikasi yang melibatkan interaksi langsung antara pengguna dan sistem melalui API. Adapun tahapan proses sebagai berikut:

1. Client Mengirim Request ke API
   Pengguna mengirimkan data suhu melalui HTTP request ke endpoint /send.
2. API Memproses dan Memvalidasi Data
   API menerima data yang dikirimkan, kemudian melakukan proses validasi sebelum diproses lebih lanjut.
3. API Mengirim Data ke Broker
   Data yang telah diproses kemudian dikirimkan ke message broker menggunakan mekanisme publish pada topik yang sama.
4. Broker Mendistribusikan Data
   Broker menerima dan mendistribusikan data ke seluruh subscriber dan API.
5. Subscriber Menampilkan Data
   Subscriber menerima data dan menampilkannya secara langsung pada terminal.
6. API Menyimpan Data
   API menyimpan data sebagai data manual terbaru untuk keperluan akses selanjutnya.
7. Client Mengakses Data melalui Endpoint `/data`
   Pengguna dapat mengakses data terbaru melalui endpoint `/data` dalam bentuk JSON.

Pada tahap awal, komunikasi bersifat sinkron karena melibatkan interaksi langsung antara client dan API. Namun, setelah data diteruskan ke broker, komunikasi berubah menjadi asinkron melalui mekanisme Publish-Subscribe.

Hal ini menunjukkan bahwa sistem mampu menggabungkan dua model komunikasi dalam satu alur yang terpadu, sehingga meningkatkan fleksibilitas sistem dalam menangani berbagai jenis interaksi.

5.4. Integrasi Model Komunikasi

Sistem yang dikembangkan tidak hanya mengandalkan satu model komunikasi, melainkan mengintegrasikan dua model komunikasi yang berbeda, yaitu Request-Response dan Publish-Subscribe, dalam satu arsitektur sistem terdistribusi yang terpadu.

Model Request-Response digunakan untuk menangani interaksi langsung antara pengguna dan sistem melalui API berbasis HTTP. Dalam model ini, pengguna dapat mengirimkan data secara manual dan menerima respon secara langsung dari sistem.

Sementara itu, model Publish-Subscribe digunakan untuk mendistribusikan data antar komponen sistem melalui perantara message broker, sehingga komunikasi dapat berlangsung secara asinkron.

Dalam implementasinya, API berperan sebagai komponen penghubung yang mengintegrasikan kedua model komunikasi tersebut. API menerima data dari pengguna melalui mekanisme HTTP, kemudian meneruskan data tersebut ke message broker menggunakan protokol MQTT. Selain itu, API juga berfungsi sebagai subscriber yang menerima data dari broker dan menyediakannya kembali kepada pengguna melalui endpoint tertentu.

Integrasi kedua model komunikasi ini memberikan sejumlah keunggulan dalam sistem yang dikembangkan. Kombinasi komunikasi sinkron dan asinkron memungkinkan sistem untuk bekerja secara lebih fleksibel, efisien, dan adaptif terhadap berbagai kebutuhan interaksi.

Adapun keunggulan integrasi model komunikasi adalah sebagai berikut:

1. Mendukung komunikasi real-time
   Sistem mampu mengirim dan menerima data secara kontinu dalam interval waktu tertentu, sehingga informasi dapat diperbarui secara cepat dan responsif.
2. Meningkatkan skalabilitas sistem (scalability)
   Penambahan komponen seperti publisher atau subscriber tidak mempengaruhi kinerja komponen lainnya secara signifikan, karena komunikasi dilakukan melalui perantara message broker.
3. Mengurangi ketergantungan antar komponen (loose coupling)
   Setiap komponen dalam sistem tidak saling bergantung secara langsung, sehingga perubahan pada satu komponen tidak mempengaruhi komponen lainnya secara keseluruhan.

5.5. Ringkasan Alur Sistem

Berdasarkan keseluruhan proses yang telah dijelaskan pada subbab sebelumnya, alur sistem dapat dirangkum sebagai suatu rangkaian proses komunikasi data yang terstruktur dalam sistem terdistribusi.

Secara umum, proses dimulai dari dua sumber utama data, yaitu data otomatis yang dihasilkan oleh publisher serta data manual yang dikirimkan oleh pengguna melalui API. Kedua jenis data tersebut kemudian dikirimkan ke message broker menggunakan protokol MQTT.

Selanjutnya, message broker berperan sebagai pusat distribusi yang meneruskan data ke seluruh komponen yang telah berlangganan (subscribe) pada topik yang sesuai. Data yang didistribusikan kemudian diterima oleh subscriber untuk ditampilkan secara langsung pada terminal, serta oleh API untuk disimpan dan disajikan kembali kepada pengguna melalui endpoint `/data`.

Dengan demikian, sistem mampu menangani aliran data secara terpusat melalui broker tanpa adanya komunikasi langsung antar komponen. Pendekatan ini tidak hanya meningkatkan efisiensi komunikasi, tetapi juga mendukung fleksibilitas dan pengembangan sistem di masa mendatang.

Secara ringkas, alur sistem dapat dijelaskan sebagai berikut:

1. Data dihasilkan oleh publisher atau dikirim oleh pengguna melalui API
2. Data dikirimkan ke message broker menggunakan protokol MQTT
3. Broker mendistribusikan data ke seluruh subscriber dan API
4. Subscriber menampilkan data secara langsung pada terminal
5. API menyimpan dan menyediakan data melalui endpoint `/data`

BAB VI
HASIL DAN ANALISIS

6.1. Hasil Simulasi

Pada bagian ini disajikan hasil dari implementasi sistem simulasi komunikasi dalam sistem terdistribusi yang telah dikembangkan. Pengujian dilakukan untuk memastikan bahwa setiap komponen sistem, yaitu publisher, message broker, subscriber, dan API, dapat berfungsi dengan baik serta saling terintegrasi sesuai dengan model komunikasi yang digunakan.

6.1.1. Hasil Eksekusi Publisher

Komponen publisher berfungsi sebagai simulasi sensor IoT yang menghasilkan data suhu secara otomatis dalam interval waktu tertentu.

```
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 21°C
```

Gambar 6.1 Hasil Eksekusi Publisher

Berdasarkan Gambar 6.1, terlihat bahwa publisher berhasil menghasilkan data suhu secara acak dan mengirimkannya secara berkala. Proses ini berlangsung secara kontinu, yang menunjukkan bahwa sistem mampu mensimulasikan pengiriman data secara real-time tanpa intervensi pengguna.

6.1.2. Hasil Penerima Data oleh Subscriber

Komponen subscriber berfungsi untuk menerima data yang didistribusikan oleh message broker.

```
publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 33°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 33°
api-1         | [API] Data AUTO: Suhu Otomatis: 33°C


publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 22°C
api-1         | [API] Data AUTO: Suhu Otomatis: 22°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 22°

publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 20°C
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 20°
api-1         | [API] Data AUTO: Suhu Otomatis: 20°C
```

Gambar 6.2 Hasil Penerimaan Data oleh Subscriber

Berdasarkan Gambar 6.2, terlihat bahwa subscriber berhasil menerima data yang dikirim oleh publisher melalui message broker. Data yang diterima ditampilkan secara langsung pada terminal, yang menunjukkan bahwa proses distribusi pesan berjalan dengan baik.

Selain itu, urutan tampilan antara publisher dan subscriber tidak selalu konsisten. Hal ini disebabkan oleh sifat komunikasi yang asinkron, di mana proses pengiriman dan penerimaan data tidak saling bergantung secara langsung.

6.1.3. Hasil Log Message Broker

Message broker dalam sistem ini menggunakan Mosquitto yang berfungsi sebagai perantara dalam proses distribusi pesan antara publisher dan subscriber. Broker menerima koneksi dari berbagai komponen sistem dan mengelola komunikasi berdasarkan topik yang digunakan.

```
broker-1      | 1776046720: mosquitto version 2.1.2 running
subscriber-1  | /app/subscriber.py:11: DeprecationWarning: Callback API version 1 is deprecated, update to lates
t version
subscriber-1  |   client = mqtt.Client()
broker-1      | 1776046721: New connection from 172.18.0.4:41087 on port 1883.
api-1         | /app/server.py:8: DeprecationWarning: Callback API version 1 is deprecated, update to latest ver
sion
api-1         |   mqtt_client = mqtt.Client()
subscriber-1  | [SUBSCRIBER] Terhubung ke broker


api-1         |  * Serving Flask app 'server'ted from 172.18.0.4:41087 as auto-5CADD27A-3964-3CB3-0163-BD6C6B6D8
publisher-1   | Publisher started...




broker-1      | 1776046721: New connection from 172.18.0.2:58641 on port 1883.
subscriber-1  | [SUBSCRIBER] Terima: Suhu Otomatis: 23°C
api-1         |  * Debug mode: off

broker-1      | 1776046721: New client connected from 172.18.0.2:58641 as auto-6090554F-8D76-297A-C42B-3FBB6D0D1
publisher-1   | /app/publisher.py:8: DeprecationWarning: Callback API version 1 is deprecated, update to latest
version
api-1         | WARNING: This is a development server. Do not use it in a production deployment. Use a productio
n WSGI server instead.


v View in Docker Desktop   o View Config   w Enable Watch   d Detach

publisher-1   |   client = mqtt.Client()
api-1         |  * Running on all addresses (0.0.0.0)


v View in Docker Desktop   o View Config   w Enable Watch   d Detach

broker-1      | 1776046721: New connection from 172.18.0.5:56239 on port 1883.
api-1         |  * Running on http://127.0.0.1:5000


publisher-1   | [PUBLISHER] Kirim: Suhu Otomatis: 23°C
api-1         |  * Running on http://172.18.0.2:5000m 172.18.0.5:56239 as auto-E65749FF-C426-A11A-1166-6D98CC274
```

Gambar 6.3 Log Message Broker

Berdasarkan Gambar 6.3, terlihat bahwa message broker berhasil dijalankan dan berada dalam kondisi aktif, yang ditandai dengan munculnya informasi versi Mosquitto yang sedang berjalan. Selain itu, log juga menunjukkan adanya beberapa koneksi yang berasal dari komponen lain dalam sistem, seperti publisher, subscriber, dan API.

Informasi seperti “New connection” dan “New client connected” menunjukkan bahwa masing-masing komponen berhasil terhubung ke broker melalui protokol MQTT. Hal ini menandakan bahwa proses inisialisasi komunikasi antar komponen dalam sistem telah berjalan dengan baik.

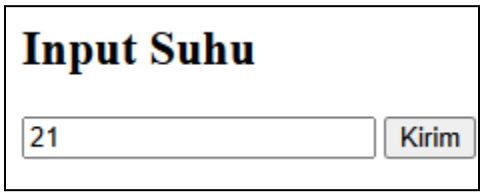
Di sisi lain, terlihat pula bahwa subscriber berhasil menerima data yang dikirim oleh publisher, yang ditunjukkan dengan munculnya log “[SUBSCRIBER] Terima: Suhu Otomatis: 23°C”. Meskipun broker tidak secara eksplisit menampilkan proses distribusi pesan dalam log, keberhasilan subscriber dalam menerima data tersebut menunjukkan bahwa broker telah menjalankan fungsinya sebagai perantara distribusi pesan dengan baik.

Perlu diperhatikan bahwa message broker bekerja secara event-driven, sehingga log yang ditampilkan hanya mencakup peristiwa tertentu seperti koneksi dan aktivitas client, bukan seluruh proses distribusi pesan secara detail.

Dengan demikian, dapat disimpulkan bahwa message broker telah berfungsi dengan baik dalam mendukung komunikasi berbasis Publish-Subscribe pada sistem yang dikembangkan.

6.1.4. Hasil Pengiriman Data Manual lewat API

Pengujian juga dilakukan dengan mengirimkan data suhu secara manual melalui API menggunakan metode HTTP.



Gambar 6.4 Input Data melalui Antarmuka Web

Berdasarkan Gambar 6.4, pengguna memasukkan nilai suhu melalui antarmuka web sebagai bentuk interaksi langsung dengan sistem. Data yang dimasukkan kemudian dikirimkan ke API menggunakan metode HTTP.

```
Pretty print
{"data_dikirim":"Suhu Input Manual: 21\u00b0C","status":"berhasil"}
```

Gambar 6.5 Hasil Pengiriman Data melalui API

Selanjutnya, berdasarkan Gambar 6.5, sistem memberikan respon berupa informasi bahwa data telah berhasil dikirim beserta nilai suhu yang dimasukkan. Hal ini menunjukkan bahwa proses komunikasi antara client dan server menggunakan model Request-Response telah berjalan dengan baik.

Pada tahap ini, data yang dikirim oleh pengguna belum ditampilkan sebagai hasil akhir sistem, melainkan masih berada pada tahap pengiriman menuju message broker untuk didistribusikan lebih lanjut.

6.1.5. Hasil Output Data pada Endpoint `/data`

Setelah data dikirim melalui API, sistem akan memproses dan menyimpan data tersebut untuk kemudian ditampilkan melalui endpoint `/data`.

```
Pretty print
{"latest_data (auto)":"Suhu Otomatis: 29\u00b0C","latest_manual":"Suhu Input Manual: 21\u00b0C"}
```

Gambar 6.6 Output Data pada Endpoint `/data`

Berdasarkan Gambar 6.6, terlihat bahwa sistem berhasil menampilkan data terbaru yang diterima, baik data yang berasal dari publisher (otomatis) maupun data yang dikirim secara manual oleh pengguna.

Berbeda dengan proses pada subbab sebelumnya, pada tahap ini data telah diproses dan disimpan oleh sistem, sehingga dapat diakses kembali melalui endpoint `/data`. Hal ini menunjukkan bahwa API tidak hanya berfungsi sebagai penerima data, tetapi juga sebagai komponen yang menyimpan dan menyediakan data kepada pengguna.

Dengan demikian, sistem mampu mengintegrasikan data dari berbagai sumber dan menyajikannya secara terpusat, yang merupakan salah satu karakteristik penting dalam sistem terdistribusi.

6.2. Analisis Perbandingan Model Komunikasi

Dalam sistem yang dikembangkan, digunakan dua model komunikasi utama, yaitu Request-Response dan Publish-Subscribe. Kedua model tersebut memiliki karakteristik yang berbeda dalam hal cara kerja, performa, serta fleksibilitas dalam sistem terdistribusi.

Tabel 6.1 Perbandingan Model Komunikasi

Aspek	Request-Response	Publish-Subscribe
Pola komunikasi	Client-Server	Broker-based
Sifat komunikasi	Sinkron	Asinkron
Ketergantungan	Tinggi (tight coupling)	Rendah (*loose coupling*)
Real-time	Terbatas	Tinggi
Skalabilitas (*scalability*)	Rendah	Tinggi
Kompleksitas implementasi	Sederhana	Lebih kompleks
Peran perantara	Tidak ada	Menggunakan message broker

Berdasarkan hasil simulasi yang telah dilakukan, model Request-Response digunakan pada komunikasi antara client dan API. Model ini memungkinkan pengguna untuk mengirimkan data dan menerima respon secara langsung. Namun, model ini memiliki keterbatasan dalam hal fleksibilitas karena adanya ketergantungan langsung antara client dan server.

Sebaliknya, model Publish-Subscribe digunakan dalam komunikasi antara publisher, message broker, dan subscriber. Model ini memungkinkan distribusi data secara real-time ke berbagai komponen tanpa adanya hubungan langsung antar komponen tersebut. Dengan adanya message broker, sistem menjadi lebih fleksibel dan memiliki tingkat scalability yang lebih tinggi.

Selain itu, konsep loose coupling pada model Publish-Subscribe memungkinkan setiap komponen sistem untuk berjalan secara independen tanpa saling bergantung secara langsung. Hal ini sangat sesuai dengan karakteristik sistem terdistribusi modern.

6.3. Analisis Kerja Sistem

Analisis kerja sistem dilakukan untuk mengevaluasi bagaimana seluruh komponen dalam sistem berinteraksi dan menjalankan fungsinya dalam lingkungan sistem terdistribusi.

Tabel 6.2 Analisis Kerja Komponen Sistem

Komponen	Fungsi	Hasil Pengujian
Publisher	Mengirim data suhu otomatis	Berjalan dengan baik, data terkirim secara berkala
Message Broker	Mendistribusikan pesan	Berfungsi sebagai perantara komunikasi
Subscriber	Menerima data dari broker	Berhasil menerima data secara real-time
API (Flask)	Menerima & menampilkan data	Berhasil mengirim, menerima, dan menampilkan data

Berdasarkan hasil pengujian, sistem menunjukkan kinerja yang stabil dalam menjalankan komunikasi antar komponen. Publisher mampu mengirimkan data secara kontinu, sedangkan subscriber berhasil menerima data tanpa hambatan yang berarti.

Message broker berfungsi sebagai pusat distribusi pesan yang menghubungkan seluruh komponen dalam sistem. Meskipun broker tidak menampilkan proses distribusi secara eksplisit dalam log, keberhasilan subscriber dan API dalam menerima data menunjukkan bahwa mekanisme distribusi berjalan dengan baik.

Selain itu, API berfungsi sebagai penghubung antara pengguna dan sistem, sekaligus sebagai subscriber tambahan yang menerima data dari broker. Hal ini memungkinkan data dapat ditampilkan kembali kepada pengguna melalui endpoint `/data`.

Sistem juga menunjukkan karakteristik komunikasi yang bersifat asinkron, yang terlihat dari urutan log antara publisher dan subscriber yang tidak selalu sama. Hal ini menunjukkan bahwa setiap komponen dapat bekerja secara independen tanpa harus menunggu komponen lain.

Secara keseluruhan, sistem telah berhasil mengimplementasikan konsep dasar sistem terdistribusi, yaitu efisiensi komunikasi, fleksibilitas, serta kemampuan untuk menangani komunikasi secara paralel.

6.4. Kesimpulan Hasil dan Analisis

Berdasarkan hasil simulasi dan analisis yang telah dilakukan, dapat disimpulkan bahwa sistem berhasil mengimplementasikan dua model komunikasi utama, yaitu Request-Response dan Publish-Subscribe, dalam satu arsitektur yang terintegrasi.

Model Request-Response terbukti efektif dalam menangani interaksi langsung antara pengguna dan sistem, terutama dalam proses pengiriman data. Namun, model ini memiliki keterbatasan dalam hal fleksibilitas dan skalabilitas.

Di sisi lain, model Publish-Subscribe menunjukkan keunggulan dalam mendukung komunikasi data secara real-time, memiliki tingkat scalability yang tinggi, serta menerapkan konsep loose coupling yang memungkinkan setiap komponen sistem berjalan secara independen.

Dengan menggabungkan kedua model komunikasi tersebut, sistem yang dikembangkan menjadi lebih fleksibel, efisien, dan mampu memenuhi karakteristik utama sistem terdistribusi modern.

BAB VII
PENGUJIAN SISTEM

7.1. Tujuan Pengujian

Pengujian sistem dilakukan untuk memastikan bahwa seluruh komponen dalam sistem simulasi komunikasi yang telah dikembangkan dapat berjalan dengan baik sesuai dengan fungsinya masing-masing. Selain itu, pengujian juga bertujuan untuk mengevaluasi performa komunikasi antar komponen dalam sistem terdistribusi, baik menggunakan model Request-Response maupun Publish-Subscribe.

7.2. Skenario Pengujian

Pengujian dilakukan dengan beberapa skenario yang mencerminkan kondisi penggunaan sistem secara nyata.

Tabel 7.1 Skenario Pengujian Sistem

No	Skenario	Tujuan
1	Menjalankan seluruh container	Memastikan sistem dapat berjalan
2	Publisher mengirim data otomatis	Menguji komunikasi Publish-Subscribe
3	Subscriber menerima data	Memastikan distribusi pesan berhasil
4	Mengirim data manual via web	Menguji Request-Response
5	Mengirim data manual via terminal	Menguji fleksibilitas sistem
6	Mengakses endpoint `/data`	Memastikan data tersimpan dan ditampilkan

7.3. Hasil Pengujian

Berdasarkan skenario yang telah dilakukan, diperoleh hasil sebagai berikut:

Tabel 7.2 Hasil Pengujian Sistem

No	Skenario	Hasil	Status
1	Menjalankan container	Semua service berjalan	Berhasil
2	Publisher mengirim data	Data terkirim tiap interval	Berhasil
3	Subscriber menerima data	Data diterima dengan baik	Berhasil
4	Input data via web	API merespon dengan benar	Berhasil

5	Input data via terminal	Data berhasil dikirim	Berhasil
6	Akses `/data`	Data ditampilkan dengan benar	Berhasil

Berdasarkan hasil pengujian yang telah dilakukan, seluruh skenario menunjukkan bahwa sistem berjalan dengan baik sesuai dengan yang diharapkan. Publisher mampu mengirimkan data secara kontinu, sementara subscriber berhasil menerima data secara real-time melalui message broker.

Selain itu, komunikasi antara client dan API menggunakan model Request-Response juga berjalan dengan baik, yang ditunjukkan dengan keberhasilan pengiriman data melalui web maupun command line. Endpoint `/data` juga mampu menampilkan data terbaru yang telah diproses oleh sistem. Hal ini menunjukkan bahwa integrasi antara model komunikasi Request-Response dan Publish-Subscribe telah berhasil diimplementasikan dalam sistem.

7.4. Kendala dan Solusi

Selama proses pengembangan dan pengujian sistem, terdapat beberapa kendala yang dihadapi, antara lain:

Tabel 7.3 Kendala dan Solusi

No	Kendala	Penyebab	Solusi
1	Data tidak muncul di endpoint `/data`	API belum subscribe ke broker	Menambahkan fungsi subscribe pada API
2	Data tidak terbaca dengan benar	Format data tidak konsisten	Menyesuaikan parsing data pada API
3	Log broker tidak menampilkan distribusi	Broker bersifat event-driven	Menggunakan subscriber sebagai indikator
4	Urutan log tidak konsisten	Sistem bersifat asinkron	Memahami konsep async dalam sistem

Kendala utama yang ditemukan adalah ketidaksesuaian antara format data yang dikirim oleh publisher dan yang diproses oleh API. Hal ini menyebabkan data tidak dapat

ditampilkan pada endpoint `/data`. Permasalahan tersebut berhasil diatasi dengan menyesuaikan mekanisme parsing data pada API.

Selain itu, sifat komunikasi yang asinkron menyebabkan urutan log antar komponen tidak selalu sama. Hal ini bukan merupakan kesalahan sistem, melainkan karakteristik dari model Publish-Subscribe.

Kendala lainnya adalah keterbatasan log pada message broker yang tidak menampilkan proses distribusi pesan secara eksplisit. Namun, hal ini dapat diatasi dengan mengamati output pada subscriber sebagai indikator keberhasilan distribusi pesan

7.5. Kesimpulan Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem telah berjalan dengan baik dan mampu memenuhi tujuan yang telah ditetapkan. Seluruh komponen sistem berhasil berfungsi sesuai dengan perannya masing-masing, baik dalam komunikasi berbasis Request-Response maupun Publish-Subscribe.

Sistem juga menunjukkan kemampuan dalam menangani komunikasi data secara real-time serta mendukung fleksibilitas dalam pengiriman dan penerimaan data.

BAB VIII
KESIMPULAN DAN SARAN

8.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem simulasi komunikasi dalam sistem terdistribusi yang telah dilakukan, dapat disimpulkan bahwa sistem berhasil mengintegrasikan dua model komunikasi utama, yaitu Request-Response dan Publish-Subscribe, dalam satu arsitektur yang terintegrasi.

Model Request-Response digunakan dalam interaksi antara pengguna dan API, yang memungkinkan proses pengiriman data secara langsung serta memperoleh respon secara cepat. Sementara itu, model Publish-Subscribe digunakan dalam komunikasi antar komponen sistem melalui message broker, yang memungkinkan distribusi data secara real-time ke berbagai komponen tanpa adanya ketergantungan langsung.

Hasil pengujian menunjukkan bahwa seluruh komponen sistem, yaitu publisher, message broker, subscriber, dan API, dapat berjalan dengan baik sesuai dengan fungsinya masing-masing. Sistem mampu mengirim, mendistribusikan, dan menampilkan data secara konsisten tanpa mengalami kendala yang signifikan.

Selain itu, sistem menunjukkan karakteristik utama dari sistem terdistribusi, yaitu fleksibilitas, efisiensi komunikasi, serta kemampuan untuk menangani komunikasi secara paralel. Konsep loose coupling yang diterapkan melalui message broker memungkinkan setiap komponen bekerja secara independen, sehingga meningkatkan scalability sistem.

Dengan demikian, sistem yang dikembangkan telah memenuhi tujuan yang diharapkan dan mampu memberikan gambaran yang jelas mengenai implementasi model komunikasi dalam sistem terdistribusi.

8.2. Saran

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan antarmuka grafis yang lebih interaktif, meningkatkan keamanan komunikasi, serta mengembangkan sistem agar dapat menangani lebih banyak node dalam skala yang lebih besar.